



Plataforma colaborativa de demostración formal de teoremas

David López · Daniel Tejeda · Emiliano Flores

CETI — Proyecto Integrador IDS · Mayo 2026

Sir Michael Atiyah y la Hipótesis de Riemann



Sir Michael Atiyah

Fields Medal 1966

Abel Prize 2004

Uno de los mayores
matemáticos del siglo XX

Septiembre 2018

Anunció una prueba de la **Hipótesis de Riemann** — uno de los 7 Problemas del Milenio, sin resolver por 160 años.

- ▶ La comunidad reaccionó de inmediato: discusiones, anotaciones, contra-argumentos
- ▶ Atiyah falleció en **enero de 2019**

Meses después el consenso fue claro: la prueba tenía un **error lógico fatal**. Llegar a ese consenso tomó **meses de colaboración asíncrona**.

¿Por qué tardó tanto?

- ▶ La revisión de pruebas es **lenta, asíncrona y subjetiva**
- ▶ No hay estándar verificable por máquina para *este paso es válido*
- ▶ Distribuir la revisión genera desacuerdos sin árbitro neutral
- ▶ El consenso depende de confianza en el criterio humano

La pregunta clave

¿Qué pasaría si la **corrección lógica** fuera tan inequívoca como un **error de compilador**?

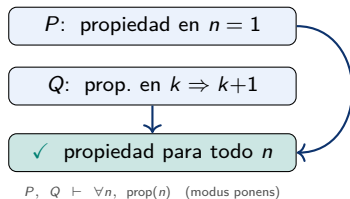
Ejemplo matemático

Teorema: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Por inducción:

- ▶ **Base:** $n = 1 \Rightarrow 1 = \frac{1 \cdot 2}{2}$ ✓
- ▶ **Paso:** si vale para k , vale para $k+1$ ✓
- ▶ **Conclusión:** vale para todo $n \geq 1$

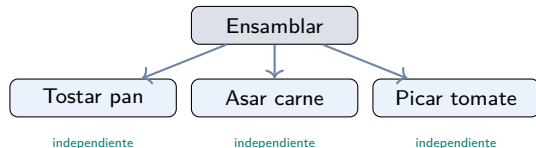
Estructura lógica pura



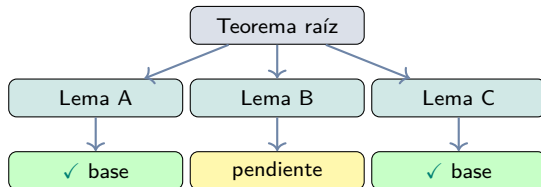
Corrección = cada paso sigue reglas admitidas.

Las Pruebas son Grafos, no Listas

Analogía: preparar una hamburguesa



Una prueba matemática funciona igual



Lemas independientes se resuelven **en paralelo**.

Automatizando la Formalización con Lean 4

En lenguaje natural

- ▶ Un humano puede *saltarse* pasos
- ▶ La validez depende de intuición compartida
- ▶ En colaboración asíncrona: los pasos omitidos son donde viven los desacuerdos

“...y por lo tanto, combinando ambos lemas, se sigue el resultado.”

En Lean 4

```
theorem sum_formula (n : Nat) :  
  Finset.sum (Finset.range (n+1)) id  
  = n * (n + 1) / 2 := by  
  induction n with  
  | zero => simp  
  | succ k ih =>  
    rw [Finset.sum_range_succ, ih]  
    ring
```

Si hay un error lógico, no compila.
No hay ambigüedad. No hay debate.

Prueba Informal vs. Prueba Formal

Prueba en lenguaje natural

Teorema: $\sqrt{2}$ es irracional.

Sea $\sqrt{2} = p/q$ con $\gcd(p, q) = 1$. Entonces $p^2 = 2q^2$, por lo que p es par. Sea $p = 2k$; entonces $q^2 = 2k^2$, así q es par. Pero $\gcd(p, q) \geq 2$. Contradicción. $\square$

¿Es correcta? Depende de que el lector acepte cada paso.

Prueba en Lean 4

```
theorem sqrt2_irrat :  
  ¬ ∃ (p q : ℤ), q ≠ 0 ∧  
    Int.gcd p q = 1 ∧ p^2 = 2 * q^2 := by  
  intro ⟨p, q, hq, hcop, heq⟩  
  have hp2 : 2 ∣ p^2 :=  
    ⟨q^2, by linarith⟩  
  have hp : 2 ∣ p :=  
    Int.Prime.dvd_of_dvd_pow  
      (Int.prime_iff.mpr (by norm_num)) hp2  
  obtain ⟨k, hk⟩ := hp  
  have heq2 : (2*k)^2 = 2 * q^2 := by rw [hk]; exact heq  
  have hq2 : 2 ∣ q^2 := ⟨k^2, by linarith⟩  
  have hq : 2 ∣ q :=  
    Int.Prime.dvd_of_dvd_pow  
      (Int.prime_iff.mpr (by norm_num)) hq2  
  have : 2 ∣ Int.gcd p q :=  
    Int.dvd_gcd hp hq  
  rw [hcop] at this  
  - this : (2 : ℤ) ∣ 1 ⇒ ⊥  
  exact absurd this (by norm_num)
```

Lean 4: Corrección Sí, Colaboración No

✓ Lo que Lean resuelve

- ▶ Árbitro neutral e infalible de corrección lógica
- ▶ Compila $\Leftrightarrow$ prueba válida
- ▶ Mathlib4: decenas de miles de teoremas verificados

✗ Lo que Lean no resuelve

- ▶ Curva de aprendizaje muy alta
- ▶ Herramienta de un solo usuario, no una plataforma
- ▶ Sin infraestructura para distribuir lemas a colaboradores
- ▶ Sin seguimiento de progreso ni gestión de contribuciones

Lean resuelve la corrección. No resuelve la colaboración.

Una plataforma web que unifica

- ▶ **DAG de prueba:** teorema raíz $\rightarrow$ lemas $\rightarrow$ base
- ▶ **Verificación automática:** cada nodo pasa por Lean 4
- ▶ **Colaboración:** GitHub como fuente de verdad; PRs para aportar
- ▶ **NL2FL:** lenguaje natural $\rightarrow$ Lean (LLM + feedback iterativo)
- ▶ **HPC:** cómputo MPI en clúster, evidencia embebida en Lean

NL2FL: ejemplo de traducción

Prueba informal (entrada):

Teorema. Si n es par, entonces $n + 2$ es par.

Prueba. Sea n par, i.e. $n = 2k$.

Entonces $n + 2 = 2k + 2 = 2(k+1)$,
que también es par. $\square$

Prueba formal (Lean 4):

```
theorem even_add_two
  (n : ℕ) (h : Even n) :
  Even (n + 2) := by
  obtain ⟨k, hk⟩ := h
  exact ⟨k + 1, by linarith⟩
```

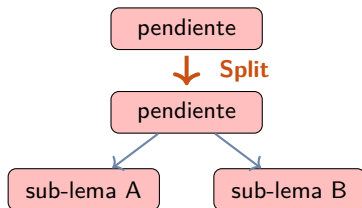
Sin necesidad de saber Lean.

Las Tres Operaciones

Cada nodo del DAG de prueba admite exactamente tres acciones:

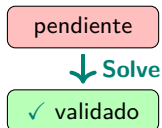
Split

Divide un nodo en sub-lemas



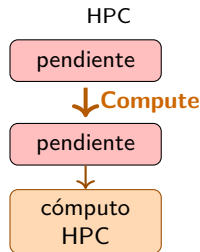
Solve

Demuestra un nodo (NL o Lean)



Compute

Añade evidencia computacional



LeanDojo / LeanCopilot (Yang et al., 2023)

- ▶ Extrae datos de Mathlib4 para entrenar modelos ML
- ▶ Búsqueda de pruebas con recuperación semántica
- ▶ Sugerencias de tácticas en tiempo real en el editor

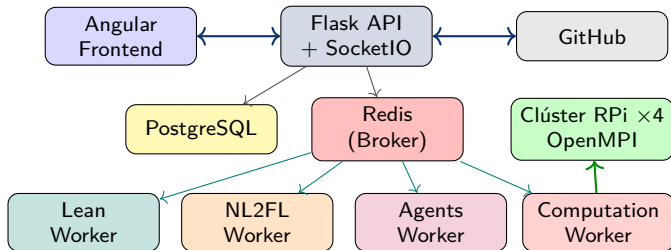
AFP — Isabelle/HOL

- ▶ Repositorio comunitario de pruebas formales
- ▶ Sin colaboración estructurada ni verificación en tiempo real

Lo que CoProof agrega

- ▶ Flujo multi-usuario con DAG y PRs
- ▶ Verificación como **requisito de aceptación**
- ▶ Pipeline NL → Lean integrado
- ▶ Cómputo HPC como artefacto formal

Ningún sistema combina colaboración estructurada, IA verificada y HPC en una sola plataforma.



`docker compose up --build`

Criterios de Éxito

Criterio	Meta	Resultado
Tasa de verificación Lean	$\geq 95 \%$	96 % ✓
Latencia promedio de verificación	$\leq 10 \text{ s}$	8,4 s ✓
Stack en un solo comando	✓	✓
Pipeline NL $\rightarrow$ Lean funcional	✓	✓
Demo E2E completo	✓	✓
Clúster HPC operativo (4 nodos)	✓	✓

Benchmark: 100 teoremas de Mathlib4 — 96 verificados, 4 timeouts ($>35 \text{ s}$), 0 errores de compilación.

Ejemplo en CoProof: Factoriones

¿Qué es un Factorión?

Un número es un **factorión** si es igual a la suma de los **factoriales de sus dígitos**.

Solo existen **4** en base 10:

$\{1, 2, 145, 40,585\}$

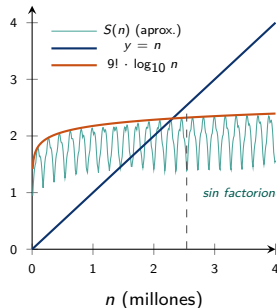
La prueba usa dos partes:

- (1) Acotar: $\exists M$ t.q. $n > M \Rightarrow S(n) < n$
- (2) Verificar exhaustivamente $n \leq M$

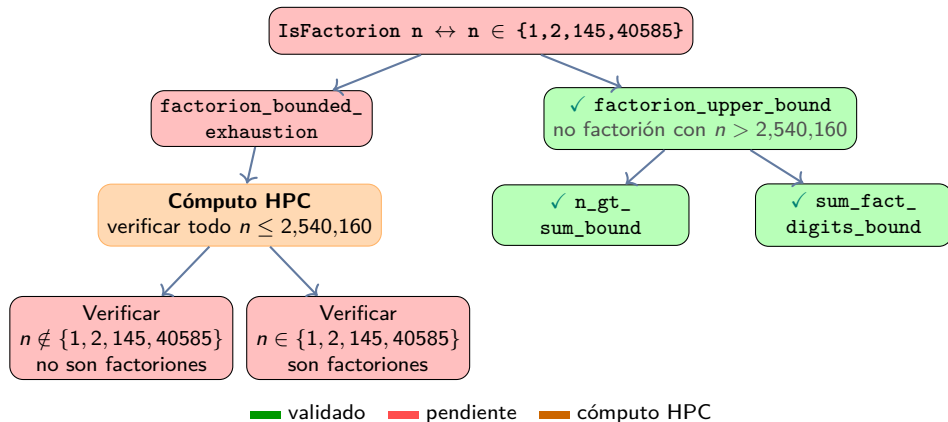
Verificando $n = 145$

$$1! + 4! + 5! = 1 + 24 + 120 = 145 \quad \checkmark$$

Cota: $S(n) \leq 9! \cdot \lfloor \log_{10} n \rfloor$ ($M = 9! \times 7 = 2,540,160$)



Factoriones: DAG de Prueba en CoProof



Flujo en vivo:

1. Crear proyecto con el teorema Factorión
2. Dividir en `upper_bound` y `exhaustion`
3. Verificar → PR automático → Merge
4. Nodo computacional → clúster Raspberry Pi

Demo en vivo

La prueba de Atiyah tenía un gap que nadie pudo formalizar por meses.

Con CoProof, ese gap habría sido un error de compilador el día uno.

